

EMIL

Evolutionary Market Intelligence Layer

HOW TO WORK ON EMIL

The operating manual for developing, extending and deploying the EMIL platform

Version: Universe Round 3 · 2 September 2026

Prepared for the EMIL owner/operator. Keep alongside GAPS.md, README.md and STYLE_GUIDE.md in the repo.

1 · What EMIL is

EMIL is a professional financial intelligence, research and (authorization-gated) trading cockpit: a multi-agent trading brain with a Bloomberg-style research terminal growing around it. It combines a customer-facing cockpit (markets, charts, correlation, news, alerts, risk, strategies, Teach EMIL) with a separate Super-Admin Command Center (CRM, data providers, API keys, feature flags, research ops, audit). The long-term vision is the EMIL Universe: OBSERVE → VERIFY → UNDERSTAND → RESEARCH → SIMULATE → HEDGE → AUTHORIZE → EXECUTE → MONITOR → LEARN, in one system.

Live product	https://serene-frangollo-a3c59c.netlify.app (also emil.abacusai.app)
Super admin	/command on the live site — admin role required, verified server-side
GitHub	https://github.com/aby777333-pixel/EMIL.git — branch main, auto-deploys are NOT relied on; deploys are triggered explicitly
Local folder	C:\Users\GIO4X\Documents\EMIL SEP 26
Database	Supabase project jnjxudsjyrftehwwdufs (Postgres + Prisma)
Netlify site ID	8d9e2863-26ee-48c4-8e0f-a4a6f0448fb1 (serene-frangollo-a3c59c)
EMIL native platform	https://dashing-hamster-0028ed.netlify.app (Raptor terminal — demo feed, labeled honestly)
Key repo docs	GAPS.md = spec gap assessment + roadmap · STYLE_GUIDE.md = design system · README.md = setup

2 · The golden rules (non-negotiable)

These come from the EMIL master specification and from hard-won lessons. Every change follows them.

- **Do not break what already works.** Inspect first, make the smallest safe change, extend instead of rewriting. Never remove working features, routes, schemas or env handling.
- **Research data ≠ execution data.** Free/keyed research feeds (Twelve Data, CoinGecko, GDELT...) power dashboards only. They must never drive an autonomous order.
- **Research availability ≠ trading availability.** EMIL may analyse an asset it cannot trade. Label trading status honestly: LIVE / VIA BROKER / COMING SOON.
- **Paper ≠ live.** Simulated anything is labeled SIMULATED / PAPER / ESTIMATED, visually unmistakable.
- **Never fake data.** Every board shows source, freshness (LIVE / DELAYED / DAILY / STALE) and fetched-at time. If data is missing, say so — an honest “unavailable” beats a fabricated number.
- **DISARM is never hidden.** While EMIL is armed, the top-bar DISARM and the sidebar EMERGENCY STOP are always visible. Kill switches are never permission-walled.
- **Secrets stay server-side.** API keys live in the DB or env, are masked in every response, and never reach browser JS. Frontend gets hasApiKey/apiKeyMasked only.
- **Every user-facing feature gets admin controls** (Command Center) and audit logging.
- **High-risk modules ship behind feature flags** (Command → Feature Flags). New flags start OFF.
- **Deploy only a clean, tested build** — the workflow in section 5, every time.

3 · Architecture

Stack

Next.js 14 App Router (pages are thin server wrappers around client components) · Prisma 6 on Supabase Postgres · NextAuth v4 credentials + JWT sessions · Tailwind + shadcn/radix UI · TradingView Lightweight Charts v5 · Abacus chat-completions for all LLM work (env ABACUSAI_API_KEY) · Netlify with @netlify/plugin-nextjs.

Surfaces

- **Cockpit** (customer-facing, cyan): dashboard, /markets, /charts, /correlation, /news, /alerts, /arm, /trades, /agents, /api-hub, /risk, /capital, /strategies, /lab, /teach, /trust, /settings. Shell: components/cockpit/shell.tsx (armed indicator, DISARM, bell, status pills).
- **Command Center** (admin-only, amber): /command + customers, connections, providers, keys, flags, research, audit. Gated in app/command/layout.tsx by a per-request DB role check — never trust the JWT role for authorization, only as a UI hint.
- **Platform API**: /api/v1/[...path] — customer API keys (SHA-256 hashed, prefix-shown-once) issued from CRM.
- **Data layer**: lib/data/hub.ts (all research fetching, caching, budgets, health) + lib/data/catalog.ts (provider catalog, seeded idempotently) + research_cache table.
- **Navigation trio**: when you add a page, update components/cockpit/nav-items.ts AND command-palette.tsx (and command/shell.tsx for admin pages). All three, every time.

Key conventions

- Pages: app/<route>/page.tsx wraps app/<route>/_components/<name>-client.tsx in CockpitShell; export const dynamic = 'force-dynamic'.
- API routes: getSession → 401; admin routes additionally requireAdmin(userId) → 403; wrap in try/catch; honest error strings; auditLog on privileged writes.
- SSR safety: never touch window.Date.now()/toLocaleString in first render — use ClientOnly, useMounted, SafeDate/SafeTime/SafeNumber (an SSR lint enforces this).
- Design tokens only (bg-primary, text-muted-foreground...) — see STYLE_GUIDE.md. DM Sans body, Plus Jakarta display, JetBrains mono for numbers.

4 · Database practice

- **Schema lives in prisma/schema.prisma** (50+ models: state/risk/consent, strategy/agents, execution, Teach knowledge graph, CRM/API keys, data hub, alerts/notifications, feature flags).
- **prisma db push HANGS on the Supabase pooler**. The working method: add models to schema.prisma, run npx prisma generate, then apply the equivalent DDL directly on Supabase (MCP execute_sql or the SQL editor). Keep both perfectly in sync.
- **Migrations are additive only** — CREATE TABLE IF NOT EXISTS, ADD COLUMN; never rename/drop production columns casually.
- **Seeding**: npx prisma db seed runs scripts/safe-seed.ts, which REFUSES to run if seed.ts contains any delete calls — do not bypass it. Seeds are idempotent upserts that never overwrite owner-entered API keys or admin toggles.
- **research_cache** is shared infrastructure: response cache + Twelve Data budget counters. It self-evicts (spent budget minutes; ~2% of writes purge entries older than 7 days).

5 · The standard change workflow

Every EMIL change, no exceptions:

1. Inspect	Read GAPS.md and the code you are about to touch. Trace dependencies and reused components. Understand before editing.
2. Extend	Smallest safe change. New feature = new files + minimal wiring, not rewrites of stable logic.
3. Schema first	If the DB changes: schema.prisma → prisma generate → DDL on Supabase (section 4) → then code.
4. Build	npx next build must exit clean. NEVER run next build while next dev is running (they share .next and corrupt each other).
5. Verify live	Run the dev server (launch config <code>emil-dev</code> , port 3470), sign in, exercise the new feature end-to-end AND the adjacent features. Check the browser console for errors.
6. Regression	Dashboard, /markets, /arm, /command load clean? Mobile layout not overflowing? Old features untouched?
7. Update GAPS.md	Move the shipped items, record fixes, keep the roadmap honest.
8. Commit + push	Descriptive commit to main on github.com/aby777333-pixel/EMIL .
9. Deploy	Trigger a REMOTE Netlify build with cache cleared: <code>npx netlify api createSiteBuild --data</code> <code>{ "site_id": "8d9e2863-26ee-48c4-8e0f-a4a6f0448fb1", "clear_cache": true }</code> Never deploy a locally-built .next from Windows.
10. Verify prod	Open the live site, sign in, spot-check the changed pages. Deployed version must match the tested build.

6 · Environment & secrets

DATABASE_URL	Supabase session-pooler URL + DB password (README has the template).
NEXTAUTH_SECRET	Long random string. Rotate the seeded one before real launch.
NEXTAUTH_URL	<code>http://localhost:3000</code> locally; the site URL in Netlify env.
ABACUSAI_API_KEY	Powers Explain, Knowledge Council, Ask EMIL, Strategy Lab. If missing, those routes return an honest 503 — set it in Netlify env too.
AWS_* (optional)	S3-compatible storage for Teach-EMIL uploads; empty disables uploads.

Provider/broker API keys are NOT env vars — the owner enters them in Command → Data Providers and the API Hub; they are stored in Postgres (currently plaintext — encrypt at rest before charging customers), masked in every API response.

Test admin: `abacus-cef4ea05@example.com` (seeded). Rotate this login and all pasted provider keys before public launch.

7 · Data Provider Hub rules

- **Free/open-first, modular, honest.** 24 catalogued providers (FRED, World Bank, IMF, OECD, BIS, Eurostat, SEC EDGAR, EIA, USDA, FAOSTAT, UN Comtrade, GDELT, Google News RSS, Open-Meteo, CoinGecko, Frankfurter, Twelve Data, Alpha Vantage, Finnhub, FMP, Nasdaq DL, OpenFIGI, GLEIF, Stooq[retired]). Each has license notes, priority, fallback, health tests.
- **All fetching goes through lib/data/hub.ts** — timeout fetch, per-provider health stamps, server-side `research_cache` (one upstream call per TTL for ALL users), stale-serve-on-failure labeled STALE.
- **Twelve Data budget: 8 credits/min on the free plan.** EMIL reserves 7/min in a shared DB counter and refuses locally with `retry-after` instead of burning doomed calls. The market board is 6 symbols (ETF proxies, labeled as proxies); watchlists cap at 8 symbols; alerts are watchlist-only so they cost zero extra credits.
- **News: GDELT primary → Google News RSS fallback**, cached 5 min per category. (Note: GDELT can appear blocked on the local ISP while fine in production.)
- **Stooq is retired upstream** (public quote CSV 404s since 2026-09-01) — kept in the catalog for reference, disabled.
- **Adding a provider:** add to `lib/data/catalog.ts` (it seeds idempotently), add a bespoke health test in `testProvider()`, fetch through `cachedFetch` with an explicit TTL, label source + freshness in the UI.

8 · Auth, roles & safety systems

- **Roles:** trader | admin (single string on User). Admin is re-verified against the DB on every privileged request (`requireAdmin`). `/login` is the only public page; `/api/signup` is open self-registration.
- **ARM/DISARM:** arming requires acknowledgements + press-and-hold; armed state shows “EMIL ARMED — LIVE TRADING ENABLED” with a permanent DISARM button. Disarm options: pause new / management only / stop automation / stop + close all.
- **Owner-only global actions:** ARM, mode change and any close-all are admin-only (EmilState is one global row and signup is open). DISARM (stop automation) and EMERGENCY STOP deliberately remain available to every signed-in user.
- **Consent + audit:** every arm/disarm/mode change writes `ConsentLog` (checkboxes, versions, method) and `AuditLog`; provider/key/flag/admin actions are audited too.
- **Feature flags:** Command → Feature Flags. `alerts_center` ON; `autonomous_trading`, `crypto_payments`, `institutional_workspaces`, `options_analytics` OFF until their engines exist. Reads cached ~30s via `lib/flags.ts` `flagEnabled()`.
- **Alert Center:** price alerts (watchlist symbols only, 20/user), race-safe triggering, in-app notifications + top-bar bell. Delivery is in-app only today — never claim email/push exist.

9 · Gotchas & landmines

- **next build during next dev corrupts .next** — stop the dev server first.
- **prisma db push hangs** on the pooler — apply DDL via Supabase directly (section 4).
- **SSR lint** (`eslint.ssr.config.mjs` — do not delete) fails builds on `window/locale/Date` in first render — use `ClientOnly` / `SafeDate` / `SafeNumber`.
- **Lightweight-charts is v5:** `chart.addSeries(CandlestickSeries, ...)` — v4 snippets (`addCandlestickSeries`) will not compile.
- **ChunkLoadErrorHandler in app/layout.tsx is required** — it works around a known chunk race; never remove layout providers.
- **YouTube ingestion (Teach)** is blocked from cloud IPs — the UI explains this honestly; captions work via `innertube` ANDROID client locally.
- **safe-seed blocks deletes** by design — never work around it; prod and dev share the same database.

- ■ **The dev server talks to the PRODUCTION Supabase DB.** Anything you create while testing is real — clean up test artifacts.
- ■ **Windows line endings:** CRLF warnings on commit are normal; do not “fix” them repo-wide.
- ■ **eslint.ignoreDuringBuilds is ON** in next.config.js — a green build does not prove lint-clean code; run the SSR lint mentally when touching first-render code.

10 · Where EMIL stands (Round 3, 2026-09-02)

Shipped and live

- Cockpit + 9-mode ARM system + guardian/risk/consent/audit + emergency stop.
- Global API Hub: 133 broker/data providers across 14 regions; per-customer credentials; India execution stack (Zerodha/Upstox/Angel/Dhan/Fyers/Breeze) with connection tests.
- Data Provider Hub (24 sources) + Global Markets terminal + charts (LW v5) + correlation engine + watchlists + news (GDELT→RSS) + server research cache.
- Teach EMIL research desk (sources→claims→concepts→hypotheses knowledge graph, Ask EMIL with citations) + Strategy Lab (estimated-data, honestly labeled).
- Command Center: CRM + plans, customer API keys, broker connections, data providers, feature flags, research ops, audit trail; Platform API /api/v1.
- Alert & Notification Center + feature flags (Round 3).

Next rounds (from GAPS.md)

- **Keys round** — add free FRED / EIA / Finnhub / FMP keys in Command → Data Providers to unlock the economic calendar, central-bank monitor, company pages and screeners.
- **Instrument master** — normalized instrument DB + symbol mapping (prerequisite for the equities terminal and global search).
- **Portfolio/exposure/scenario layer** — consolidated multi-broker portfolio, exposure map, scenario + hedge simulators.
- **Commerce round** — payment gateway, billing portal, wallet (behind flags).
- **Execution safety round** — circuit breakers on data health, execution protections, real backtesting.

Before charging customers (hard checklist)

- Rotate the seeded test-admin login and NEXTAUTH_SECRET.
- Encrypt provider/broker credentials at rest (currently plaintext in Postgres).
- Rate-limit /api/signup and login; add 2FA for admin.
- Real payment gateway before any paid plan is advertised as purchasable.
- Per-user trading state if EMIL ever manages more than the owner's account.

Final word: EMIL's edge is honesty. Every number says where it came from, how fresh it is, and what EMIL cannot do yet. Protect that above every feature. Inspect first. Extend carefully. Test thoroughly. Deploy only when stable.